

01-docker-realtime.md

First: What you got wrong

1. “Monolithic has only one database”

Not always.

Many monolith apps use **multiple databases**. The key point is **the application is deployed as one unit**, not the database count.

2. “Microservices = one server per service”

Not required.

You can run **many microservices on the same server**.

3. “Docker solves microservices server problem”

Docker doesn't reduce servers magically.

Docker just **packages apps so they run consistently**. You still need infrastructure.

Your assumption:

microservices → many servers → docker → single server

Reality:

microservices → **many services** → usually **containers** → often **orchestrated with Kubernetes**

Correct Concept (Simple)

Monolithic Architecture

- One big application
- All features inside the same codebase
- Deployed together

Example features:

- payments
- movie booking
- banking
- notifications

All inside **one application**

Microservices Architecture

- Each feature is its **own small application**
- They communicate via **API**

Example services:

- payment-service
- movie-service
- ticket-service
- notification-service

Each runs **independently**

ELI5 (Explain Like I'm 5)

Imagine a **restaurant kitchen**.

Monolithic Kitchen

One big kitchen.

One chef cooks:

- pizza
- burger
- pasta
- dessert

If the **chef gets sick**,
the whole restaurant stops.

Microservices Kitchen

Different chefs.

Chef 1 → pizza

Chef 2 → burger

Chef 3 → dessert

If the **pizza chef is sick**,
burger and dessert still work.

ELI10 (Explain Like I'm 10)

Think about a **school exam system**.

Monolithic System

Everything runs in **one big program**.

Login

Payments

Movie booking

Bank transfer

Notifications

If you change **one small thing**,
you must **restart the whole system**.

If it crashes → **everything stops**.

Microservices System

Each feature is a **separate small program**.

Login Service

Payment Service

Booking Service

Notification Service

If **payment service crashes**,
login and booking still work.

You only restart **that service**.

Why companies moved to microservices

Companies like **Netflix**, **Amazon**, and **Uber** moved to microservices because:

- independent deployments
- faster development
- easier scaling
- fault isolation

The Hidden Problem (people don't talk about)

Microservices also bring **serious complexity**.

You suddenly need:

- service discovery
- API gateway
- distributed logging
- monitoring
- container orchestration
- CI/CD for many services

Tools usually involved:

- **Docker**
- **Kubernetes**
- **Prometheus**
- **Grafana**
- **Istio**

Brutal Reality (important for DevOps)

People think:

 microservices = better

Wrong.

For **80% of companies**, a **well-designed monolith is actually better**.

Microservices make sense when:

- huge scale
- hundreds of developers
- independent teams
- frequent deployments

Real architecture evolution

Most companies go like this:

Monolith

↓

Modular Monolith

↓

Microservices

Jumping directly to microservices often creates a **distributed mess**.

Uses Containers Instead of Virtual Machines — ELI5

Imagine you have **one big house**.

That house is your **computer**.

Virtual Machines (Old Way)

If you want 3 separate apps, you build **3 separate small houses inside the big house**.

Big House (Computer)

|

| Small House | Small House | Small House |

| VM1 | VM2 | VM3 |

Each small house needs its own:

- kitchen
- bathroom
- electricity

This is like **Virtual Machines**.

Every VM needs its own **full operating system**, so it uses **more memory and resources**.

Tools like **VMware** work this way.

Containers (Docker Way)

Now imagine instead of building new houses, you create **separate rooms inside the same house**.

Big House (Computer)

|

| Room 1 | Room 2 | Room 3 |

Each room has:

- its own people
- its own work

But **they share the same kitchen and electricity**.

This is how containers work with **Docker**.

Containers share the **same operating system**, but keep applications separate.

Why Containers Are Better

Because containers share the same OS:

- They start **very fast**
- They use **less RAM**
- You can run **many apps on one server**

One-Line Understanding

Containers in **Docker** are like **separate rooms in one house**, while Virtual Machines are like **building separate houses inside the same land**.

Uses Containers Instead of Virtual Machines — ELI5

Imagine you have **one big house**.

That house is your **computer**.

Virtual Machines (Old Way)

If you want 3 separate apps, you build **3 separate small houses inside the big house**.

Big House (Computer)

|

| Small House | Small House | Small House |

| VM1 | VM2 | VM3 |

Each small house needs its own:

- kitchen
- bathroom
- electricity

This is like **Virtual Machines**.

Every VM needs its own **full operating system**, so it uses **more memory and resources**.

Tools like **VMware** work this way.

Containers (Docker Way)

Now imagine instead of building new houses, you create **separate rooms inside the same house**.

Big House (Computer)

|

| Room 1 | Room 2 | Room 3 |

Each room has:

- its own people
- its own work

But **they share the same kitchen and electricity**.

This is how containers work with **Docker**.

Containers share the **same operating system**, but keep applications separate.

Why Containers Are Better

Because containers share the same OS:

- They start **very fast**
 - They use **less RAM**
 - You can run **many apps on one server**
-

One-Line Understanding

Containers in **Docker** are like **separate rooms in one house**, while Virtual Machines are like **building separate houses inside the same land**.

1 Docker Architecture (Simple Diagram)

Developer

|

Docker Client

|

Docker Host

|

Docker Daemon

|

| Containers | Images | Volumes | Network |

|

Docker Registry (Docker Hub)

2 ELI5 (Explain Like I'm 5)

Imagine you want to cook food.

You (Developer)

|

Kitchen Button Panel (Docker Client)

|

Chef (Docker Daemon)

|

Kitchen (Docker Host)

|

Food Recipes (Images)

|

Prepared Food (Containers)

Parts Explained

Docker Client

This is where you type commands like:

`docker build`

`docker run`

`docker pull`

You are telling Docker what to do.

Docker Host

This is the **computer where Docker is installed**.

Docker Daemon

Think of this as the **chef in the kitchen**.

The daemon runs in the background and does the real work:

- builds images
 - runs containers
 - manages networks
 - manages volumes
-

Docker Registry

This is like a **storehouse for recipes**.

Example registry: **Docker Hub**

You can:

- download images (docker pull)
 - upload images (docker push)
-

3 ELI10 (Beginner Technical Explanation)

Docker works using a **client–server model**.

Step 1 — Developer gives command

Example:

```
docker run nginx
```

This command is executed from the **Docker Client**.

Step 2 — Client sends request

The Docker Client sends the request to the **Docker Daemon**.

Client → Daemon

Step 3 — Daemon performs the task

The daemon may:

- download an image from **Docker Hub**
 - create a container
 - start the container
-

Step 4 — Container runs application

The container runs the application in isolation.

Example containers:

Container 1 → Nginx

Container 2 → Bank App

Container 3 → Movie App

4 Feynman Explanation (Deep but Simple)

Docker architecture has **three main layers**.

1 Docker Client

This is the **interface used by the developer**.

Commands:

docker build

docker pull

docker run

docker push

The client **does not run containers itself**.

It only sends instructions.

2 Docker Host

The Docker Host is the **machine where Docker is installed**.

Inside the host runs the **Docker Daemon (dockerd)**.

The daemon is responsible for managing:

- **Images** → templates for containers
- **Containers** → running applications

- **Volumes** → persistent storage
 - **Networks** → container communication
-

3 Docker Registry

A Docker Registry stores Docker images.

The most common registry is **Docker Hub**.

Example workflow:

Developer builds image

↓

Image pushed to Docker Hub

↓

Another server pulls the image

↓

Container runs from that image

5 Complete Docker Workflow

Developer

|

docker build

docker run

docker pull

|

Docker Client

|

Docker Daemon

|

| Images | Containers | Network |

|

Docker Hub (Registry)

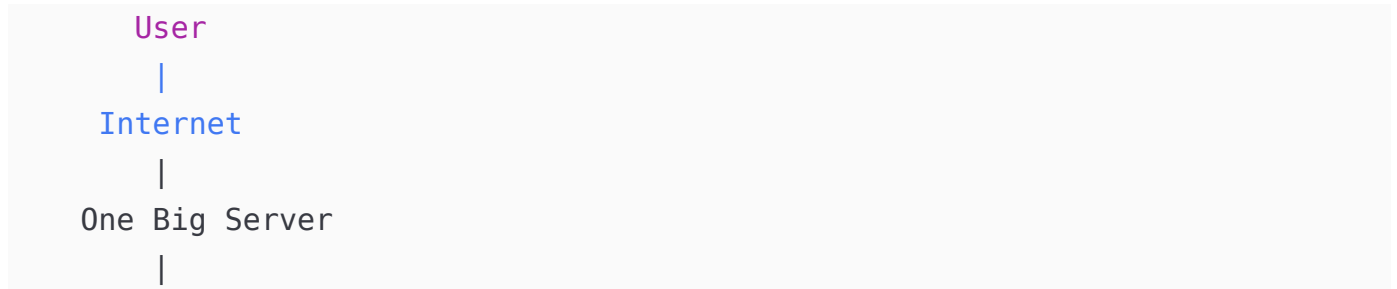
6 One-Line Summary

In **Docker**, the **client sends commands**, the **daemon executes them**, and **images are stored in registries like Docker Hub** to create and run containers.

Monolithic vs Microservices Architecture

1 Monolithic Architecture (Single Big Application)

Diagram



| Frontend | Backend | Database |

Everything runs as **one single application**.

Example structure:

Movie App

- |— Login
- |— Payment
- |— Movies
- |— Notifications
- |— Admin

All features are inside **one codebase and one deployment**.

2 ELI5 (Monolithic)

Imagine a **big school bag**.

One Bag

- |
- Books
- Lunch
- Water
- Shoes
- Toys

Everything is in **one bag**.

If one thing breaks, you must open the **whole bag**.

That is **Monolithic architecture**.

3 Problems with Monolithic Architecture

When the application grows:

1 Hard to scale

If **only login traffic increases**, you still must scale the **whole app**.

2 Slow deployments

Small change → redeploy entire system.

3 One bug can crash everything

Payment bug

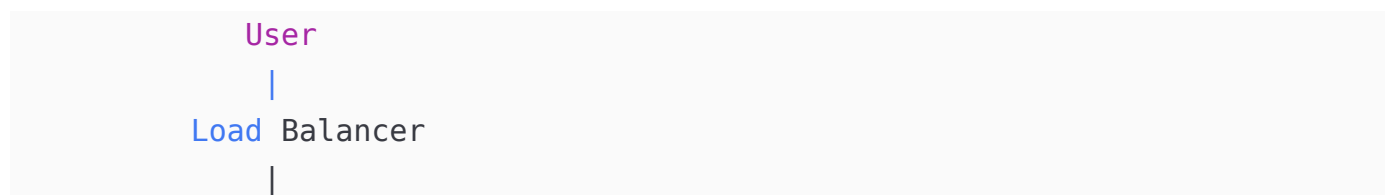
↓

Whole application crashes

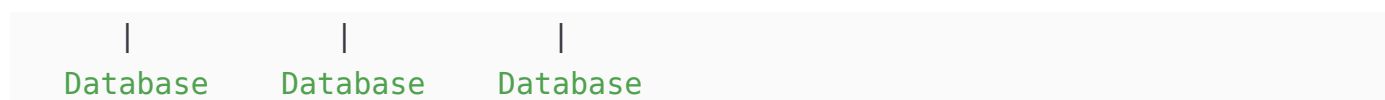
4 Microservices Architecture

Instead of one big application, the system is broken into **small independent services**.

Diagram



| Movie Service | Payment | Auth Service |



Each service runs **independently**.

5 ELI5 (Microservices)

Instead of **one big bag**, imagine **many small boxes**.

Toy Box
Book Box
Lunch Box
Water Bottle

If one box breaks, the others still work.

That is **Microservices**.

6 ELI10 (Technical View)

A system is divided into services like:

User Service
Payment Service
Movie Service
Notification Service

Each service:

- runs independently
- has its own database
- can be deployed separately

Communication happens using APIs.

User Service → Payment Service → Notification Service

7 Where Docker Fits

Microservices require **many small applications**.

Running them directly on servers becomes messy.

Example:

Server
|
Auth App
Movie App
Payment App
Notification App

Dependency conflicts appear.

So we use **containers with Docker**.

Server

|

| Auth Container |

| Movie Container |

| Payment Container |

| Notification Container |

Each container runs **isolated dependencies**.

Images are stored in **Docker Hub**.

8 Real DevOps Architecture

Large companies run hundreds of services.

Example:

Internet

|

Load Balancer

|

API Gateway

|

| Auth Service |

| Order Service |

| Payment Service |

| Notification Service |

|

Databases

Each service often runs in containers managed by orchestration systems.

9 Feynman Explanation

Monolithic architecture bundles **all functionality into a single deployable unit**, which simplifies development initially but becomes difficult to scale, maintain, and deploy as complexity grows.

Microservices architecture decomposes the system into **independent services**, each responsible for a specific domain. This enables independent scaling, faster deployments, and fault isolation, but introduces complexity in communication, monitoring, and deployment.

Containerization tools like **Docker** make microservices practical by packaging each service with its dependencies and allowing them to run consistently across environments.

10 One-Sentence Summary

Monolith = **one big application**

Microservices = **many small independent services**

Containers (Docker) make microservices **manageable and portable**.